AUDITORIA DE SEGURANÇA · CÓDIGO-FONTE

Relatório de Auditoria de Segurança — NaicoTech Store

Isolamento de dados, permissões, IDOR, segredos e XSS

Data	10/10/2026
Versão auditada	commit 0a161e6 (branch main)
Escopo	backend/ (NestJS: 11 controllers, 51 handlers, serviços, guards, middlewares), frontend/ (React: páginas, componentes, cliente da API, server.mjs, middleware.js, CSP), prisma/, scripts/, docker-compose.yml, render.yaml, vercel.json, .github/workflows/ci.yml, docs/, histórico git completo e bundle compilado (frontend/dist).
Stack detectada	TypeScript · NestJS 12 · Prisma 7 + PostgreSQL 16 · sessão JWT HS256 em cookie httpOnly + 2FA TOTP para admin · React 19 + Vite · deploy Render (API) + Vercel (front com proxy) · Docker Compose (dev) · GitHub Actions (CI).
Resultado	**9 achados**: nenhum crítico ou alto · 1 médio · 4 baixos · 4 informativos.
Situação	**8 corrigidos e 1 aceito** (F7, chave pública por design) no branch `fix/auditoria-seguranca`, com testes automatizados para cada correção.

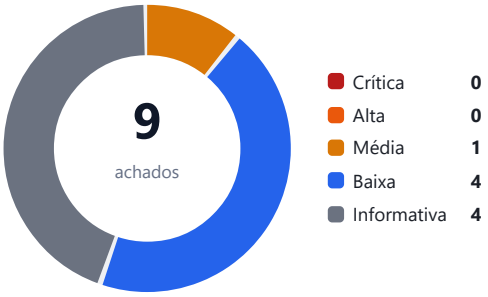
Nota metodológica — como cada categoria foi mapeada para a stack

Categoria	Equivalente nesta stack e o que foi verificado
1. Banco sem tranca	Sem Supabase/RLS. Loja single-tenant: o isolamento é por **dono**, via filtro manual `userId` nas queries Prisma e `req.user.sub` vindo do guard. Todas as queries de listagem, exportação e agregação foram conferidas.
2. Permissão no navegador	Gates de papel do React (`isAdmin`, `user.role`) cruzados um a um com as rotas do NestJS e seus guards (`@AdminOnly`, `@Roles`, `RolesGuard`, `AdminStepUpGuard`), incluindo exceções por papel dentro dos serviços.
3. IDOR	Todos os handlers com `:id` no path, query ou body (inclusive `addressId` e `productId`) verificados quanto à posse no serviço.
4. Chaves expostas	Grep de padrões de chave/token no código, configs, CI, scripts e docs; leitura de `validateEnv()`; `git log --all -p`; varredura do bundle Vite; conferência (sem imprimir valores) de que os .env locais não reaproveitam valores públicos.
5. XSS	Front: sinks de HTML, `href/src` dinâmicos, `window.open`, eval, CSP. Back: HTML dos e-mails (nodemailer) e respostas. Não há templates server-side nem markdown.

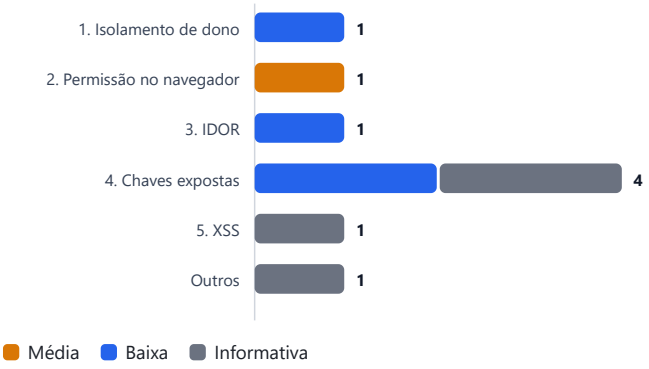
Resumo executivo

0	0	1	4	4
Crítica	Alta	Média	Baixa	Informativa

Achados por severidade



Achados por categoria (empilhado por severidade)



Leitura geral: a base é defensiva e consistente — sessão revalidada no banco a cada requisição, painel admin com 2FA obrigatória e auditoria, posse verificada em todas as rotas de cliente, nenhum segredo real no código ou no histórico e nenhum sink de XSS. O risco mais relevante (médio) é uma rota de moderação que aceita o papel ADMIN sem passar pela 2FA, contornando a principal defesa do painel. **Situação:** todos os achados acionáveis foram corrigidos no branch `fix/auditoria-seguranca`; as seções abaixo descrevem o estado auditado (commit 0a161e6 (branch main)).

Pontos fracos (riscos centrais)

Exceções por papel ("role !== 'ADMIN'") dentro de rotas de cliente contornam a política do painel (2FA + auditoria). É o padrão por trás de F1 e F2: o privilégio existe em dois lugares, e só um deles está blindado.

Sem RLS: o isolamento por dono depende de cada query lembrar o filtro userId. Hoje todas lembram, mas não há segunda camada se uma rota nova esquecer.

A validação de startup confia no formato dos segredos, não na origem: valores de teste públicos passariam em produção (F4).

Infra de desenvolvimento exposta na rede local por padrão (F6).

Pontos fortes (verificados no código)

Autenticação. Sessão em cookie httpOnly + SameSite=Strict + Secure em produção; papel, tokenVersion e e-mail verificado relidos do banco a cada requisição (logout e rebaixamento valem na hora); typ "session" vs "admin" impede usar um token pelo outro.
`auth/session.ts:12-19 · guards/jwt-auth.guard.ts:52-73`

Painel admin. @AdminOnly() = JwtAuthGuard + RolesGuard + AdminStepUpGuard (2FA TOTP, 30 min) + AuditInterceptor. Aplicado em todas as escritas de produtos/categorias e em todas as rotas /admin do painel.
`admin/admin-only.decorator.ts:12-17 · products.controller.ts:30-46 · categories.controller.ts:30-46 · admin.controller.ts:130-195`

Posse (IDOR). Pagamento confere o dono (404 para pedido alheio); endereços são removidos/lidos com where {id, userId}; conta, pedidos e endereços usam só req.user.sub; avaliação exige compra própria paga.
`orders.service.ts:163 · addresses.service.ts:52, 59 · account.controller.ts:37, 44 · reviews.service.ts:16-25`

Listagens. GET /orders, /addresses e a exportação LGPD filtram por userId; produtos públicos usam select que omite costPrice; endereços só voltam mascarados.
`orders.service.ts:143-150 · addresses.service.ts:20-34 · account.service.ts:37-49 · products.service.ts:7-19`

Integridade de valores. Preço sempre do banco, frete recotado no servidor, baixa de estoque atômica, webhook com HMAC + tolerância de 15 min, reconsulta ao Mercado Pago e checagem do valor pago.
`orders.service.ts:82, 105-115 · payments.service.ts:78-93 · payments.controller.ts:55, 71`

Mass assignment / CSRF. ValidationPipe com whitelist + forbidNonWhitelisted; CSRF por Origin + exigência de JSON; CORS restrito às origens configuradas.
`app.setup.ts:38-52 · security/csrf.middleware.ts:14-29`

Segredos. Nenhum segredo real no código nem no histórico git; .env ignorado; startup recusa placeholders, JWT < 32 caracteres e chaves AES repetidas; render.yaml usa generateValue/sync:false; sourcemap desligado no build.
`config/env.ts:2-43 · .gitignore:1 · render.yaml:25-61 · vite.config.ts:30`

XSS. CSP com nonce por resposta e frame-ancestors none; imagens de produto só https (@IsUrl); campos de endereço bloqueiam < e >; link de rastreio com encodeURIComponent; e-mails escapam nome, parágrafos e URL do botão.
`frontend/csp.mjs:19-35 · create-product.dto.ts:44-49 · shipping/address.ts:17 · utils/format.ts:53-54 · mail.service.ts:4-10, 149-172`

Dados em repouso. Endereços, segredos TOTP e IPs criptografados com AES-256-GCM, chaves separadas e AAD amarrando o texto ao dono.
`security/secret-box.ts:10-35 · shipping/address.ts:115-127`

Achados detalhados por categoria

Categoria 1 — Banco sem tranca

Mecanismo de isolamento: loja single-tenant (sem organização/workspace). O isolamento é por dono, feito à mão: filtro userId nas queries Prisma + guards NestJS. Não há RLS no Postgres (um único usuário de banco), então a proteção depende 100% do código da aplicação.

Severidade	Arquivo:linha	Descrição
Baixa F3 Corrigido	backend/src/reviews/ reviews.service.ts:42-53 (linha 49)	**Listagem pública de avaliações expõe o nome completo dos compradores** A rota pública GET /reviews/product/:productId devolve user.name inteiro de outros usuários; o frontend o exibe (ProductPage.tsx:217). O restante do projeto minimiza dados pessoais (endereço mascarado, só o primeiro nome no Header). `select: { id: true, rating: true, comment: true, createdAt: true, user: { select: { name: true } }, },` **Explorabilidade:** Qualquer visitante, sem login, lista o nome completo de quem comprou cada produto (associa pessoa ↔ compra). Exposição de dado pessoal (LGPD, minimização), não de credencial. Corrigido: a rota pública devolve só o nome reduzido (ex.: "Ana L.").

Categoria 2 — Permissão definida no navegador

Gates do frontend: AdminPage.tsx:20/48 (isAdmin) e Header.tsx:98 (link Admin). Todas as 21 chamadas de adminApi (endpoints.ts:87-121) foram cruzadas com o backend: cada uma tem @AdminOnly() ou @Roles('ADMIN')+RolesGuard. O único furo é uma rota sem UI no front (F1).

Severidade	Arquivo:linha	Descrição

Média
F1
Corrigido
	backend/src/reviews/
	reviews.controller.ts:32-36
	reviews.service.ts:58-62
		Exclusão de avaliações por ADMIN sem 2FA e sem auditoria
		DELETE /reviews/:id aceita role ADMIN só com o cookie de sessão comum (JwtAuthGuard). Não passa pelo AdminStepUpGuard (2FA) nem pelo AuditInterceptor, que protegem todas as outras ações administrativas. Os IDs das avaliações são públicos em GET /reviews/product/:productId.
		`@Delete(':id')`
		`@UseGuards(JwtAuthGuard)`
		`remove(@Req() req, @Param('id', ParseUUIDPipe) id: string) {`
		`  return this.reviewsService.remove(id, req.user.sub, req.user.role);`
		`}`
		`// reviews.service.ts:59`
		`if (review.userId !== userId && role !== 'ADMIN') { throw new ForbiddenException(...) }`
		Explorabilidade: Quem obtiver só a senha de um admin (phishing, reuso de senha) — exatamente o cenário que a 2FA obrigatória existe para barrar — apaga qualquer avaliação da loja, sem deixar registro em AdminAuditLog.
		Corrigido: DELETE /reviews/:id só para o autor; moderação em DELETE /admin/reviews/:id com @AdminOnly() (2FA + auditoria).

Categoria 3 — IDOR

Todos os 51 handlers de 11 controllers foram percorridos. Todos os :id usam ParseUUIDPipe; nenhuma rota aceita userId vindo do cliente.

Severidade	Arquivo:linha	Descrição

Baixa
F2
Corrigido
	backend/src/orders/
	orders.service.ts:152-158
		GET /orders/:id libera pedido de qualquer cliente para ADMIN sem 2FA
		O dono é verificado, mas há um bypass por papel: role ADMIN lê qualquer pedido com a sessão comum, sem a 2ª etapa e sem auditoria. O painel não usa essa rota (usa /admin/orders), então o bypass é desnecessário.
		`const order = await this.prisma.order.findUnique({ where: { id }, ... });`
		`// 404 também para pedido de outra pessoa: não confirma que o ID existe.`
		`if (!order || (order.userId !== userId && role !== 'ADMIN')) {`
		`  throw new NotFoundException('Pedido não encontrado');`
		`}`
		Explorabilidade: Exige senha de admin + o UUID do pedido (v4, não enumerável; aparece em links de e-mail como /pedidos/<id>). Expõe itens, valores, frete, código de rastreio e o resumo mascarado do endereço.
		Corrigido: exceção de ADMIN removida de OrdersService.findOne; a rota de cliente atende só o dono.

Categoria 4 — Chaves expostas

Varridos: código, .env.example, docker-compose, render.yaml, vercel.json, CI, scripts, docs, histórico git completo (git log --all -p) e o bundle compilado (frontend/dist).

Severidade	Arquivo:linha	Descrição

Baixa
F4
Corrigido
	`backend/src/config/env.ts:2, 17-60`	**Validação de startup não rejeita as chaves de teste commitadas nem chaves de baixa entropia**
		validateEnv() bloqueia só os placeholders do .env.example, tamanho mínimo e formato hex. Os valores fixos de backend/test/setup-env.ts (JWT_SECRET, chaves 0123.../fedc.../0011...) passam em todas as checagens. Em produção, PROXY_SECRET e SMTP_HOST também não são exigidos.
		`const PLACEHOLDERS = ['GERE_UM', 'TROQUE_POR', 'seu-access-token', 'chave-secreta', 'cole-o-token'];`
		`...`
		`if (jwtSecret.length < 32) fail(...)`
		`if (!/^[0-9a-f]{64}$/i.test(totpKey)) fail(...)`
		Explorabilidade: Condição: alguém copia os valores dos testes para o .env de produção. Aí o JWT_SECRET é público (forja de sessão de qualquer usuário, inclusive token typ "admin") e as chaves AES de TOTP/endereço/IP também. Verificado: o backend/.env local NÃO usa esses valores.
		Corrigido: validateEnv() recusa segredos de teste (atuais e antigos), chaves repetitivas, e exige SMTP_HOST em produção e PROXY_SECRET no modo demo.

Informativa
F5
Corrigido
	`backend/test/setup-env.ts:8-31`
	`.github/workflows/ci.yml:21, 32`
		Credenciais de teste fixas no repositório
		Segredos falsos dos testes e a senha do Postgres efêmero do CI estão em texto puro. Não dão acesso a nada real; o risco é só alimentar o F4.
		`process.env.JWT_SECRET = 'segredo-de-teste-com-mais-de-32-caracteres-0123456789';`
		`process.env.TOTP_ENCRYPTION_KEY = '0123456789abcdef...';`
		`# ci.yml`
		`POSTGRES_PASSWORD: senha-so-do-ci`
		Explorabilidade: Não explorável sozinho (banco do CI é descartado ao fim do job; chaves de teste não existem em produção).
		Corrigido: os testes usam segredos derivados de rótulos públicos, recusados fora de NODE_ENV=test. A senha do Postgres efêmero do CI foi mantida (não protege nada real).

Baixa
F6
Corrigido
	`docker-compose.yml:11-12, 21-23`	**Postgres e Mailpit de dev publicados em todas as interfaces de rede**
		As portas 5433 (Postgres), 1025 (SMTP) e 8025 (interface web do Mailpit, sem autenticação) são publicadas em 0.0.0.0. O Mailpit guarda os e-mails de verificação e de redefinição de senha com tokens de uso único.
		`ports:`
		`  - "5433:5432"`
		`...`
		`ports:`
		`  - "1025:1025"`
		`  - "8025:8025"`
		Explorabilidade: Condição: máquina de dev numa rede compartilhada (Wi-Fi público, coworking) com o firewall permitindo. Qualquer um na rede abre http://<ip>:8025, lê os links de /redefinir-senha e toma contas de dev (inclusive a de admin local).
		Corrigido: portas publicadas só em 127.0.0.1.

Informativa F7 Aceito	`frontend/dist/assets/index-*.js` `frontend/src/components/CardPaymentForm.tsx:6`	**Public Key do Mercado Pago embutida no bundle (esperado)** Única chave encontrada no bundle compilado: VITE_MP_PUBLIC_KEY (APP_USR-...). É pública por design (só tokeniza cartões no navegador). Nenhum segredo de servidor vai para o bundle: PROXY_SECRET/API_URL são lidos só no middleware do Vercel. `const publicKey = import.meta.env.VITE_MP_PUBLIC_KEY` **Explorabilidade:** Não explorável. Registrado para documentar a verificação do bundle. Aceito: chave pública por design, nada a corrigir.

Categoria 5 — Inputs sem tratamento (XSS)

Nenhum innerHTML, dangerouslySetInnerHTML, eval ou new Function. Não há lib de sanitização porque nenhum ponto renderiza HTML: o React escapa tudo. HTML só é montado nos e-mails, com escapeHtml().

Severidade	Arquivo:linha	Descrição

Informativa
F8
Corrigido
	`frontend/src/pages/admin/OrdersTab.tsx:99`
	`backend/src/shipping/shipments.service.ts:165`
		URL de impressão da etiqueta aberta sem validar o protocolo
		A URL devolvida pela API do Melhor Envio é repassada ao navegador e aberta com window.open sem checar se é https.
		`const { url } = await guard(adminApi.printLabel(order.id))`
		`window.open(url, '_blank', 'noopener,noreferrer')`
		Explorabilidade: Só explorável se a API do Melhor Envio (terceiro confiável, via HTTPS) for comprometida e devolver uma URL javascript:/data:. Defesa em profundidade.
		Corrigido: backend recusa URL não-https (502) e o painel só abre https.

Fora das 5 categorias

Severidade	Arquivo:linha	Descrição
Informativa F9 Corrigido	`backend/src/auth/auth.service.ts:243-246`	**E-mail completo no log quando o envio falha** Fora das 5 categorias, anotado por contrariar a política do próprio projeto (mail.service.ts:12 e :176 mascaram o e-mail no log). `this.logger.error(`Falha ao enviar e-mail de ${kind} para ${email}`, error as Error)` **Explorabilidade:** Quem tiver acesso aos logs vê e-mails de clientes em claro (dado pessoal). Corrigido: e-mail mascarado no log (maskEmail).

Recomendações priorizadas

P1 **Fechar a exclusão de avaliações sem 2FA (F1)**
Tirar o bypass de ADMIN de DELETE /reviews/:id (só o autor apaga) e criar DELETE /admin/reviews/:id com @AdminOnly(), que já traz 2FA e auditoria.

P2 **Remover o bypass de ADMIN em GET /orders/:id (F2)**
Rota de cliente atende só o dono. Se o painel precisar ver um pedido, criar GET /admin/orders/:id com @AdminOnly().

P2 **Endurecer a validação de segredos (F4, F5)**
Lista de valores proibidos com os segredos de teste, checagem mínima de entropia (bytes distintos) e, com NODE_ENV=production, exigir PROXY_SECRET e SMTP_HOST.

P3 **Prender a infra de dev ao localhost (F6)**
Publicar as portas como "127.0.0.1:5433:5432", "127.0.0.1:1025:1025" e "127.0.0.1:8025:8025".

P3 **Minimizar o nome nas avaliações públicas (F3)**
Devolver só o primeiro nome (ou "Ana S.") na rota pública, mantendo o nome completo fora da resposta.

P4 **Higiene: log e URL de etiqueta (F8, F9)**
Usar maskEmail() no log do AuthService e validar no backend que a URL de impressão é https antes de devolvê-la.

ISSUES PARA O GITHUB

Uma issue por achado acionável; achados triviais do mesmo tema foram agrupados (F4+F5 e F8+F9). F7 não gera issue (chave pública por design). Copie o conteúdo entre os delimitadores. **Todas já foram resolvidas no branch `fix/auditoria-seguranca`**; o texto fica como registro do problema e dos critérios de aceite usados.

--- ISSUE 1 ---

```
# [Segurança] Exclusão de avaliações por ADMIN sem verificação em duas etapas nem auditoria

**Labels sugeridas:** `security`, `severity:medium`

## Descrição
`DELETE /reviews/:id` permite que uma conta com `role = ADMIN` apague a avaliação de **qualquer** usuário usando apenas o cookie de sessão comum. A rota usa só o `JwtAuthGuard`, então não passa pelo `AdminStepUpGuard` (2FA TOTP) nem pelo `AuditInterceptor`, que protegem todas as outras ações administrativas (`@AdminOnly()`).

Os IDs das avaliações são públicos em `GET /reviews/product/:productId`.

## Por que é explorável
A 2FA obrigatória do painel existe para que **só a senha** de um admin não baste. Aqui basta: quem obtiver a senha (phishing, reuso de senha) faz login, lista os IDs pela rota pública e apaga todas as avaliações. Nada é registrado em `AdminAuditLog`.

## Evidência
`backend/src/reviews/reviews.controller.ts:32-36`
```ts
@Delete(':id')
@UseGuards(JwtAuthGuard)
remove(@Req() req: AuthenticatedRequest, @Param('id', ParseUUIDPipe) id: string) {
  return this.reviewsService.remove(id, req.user.sub, req.user.role);
}
```
`backend/src/reviews/reviews.service.ts:58-62`
```ts
if (review.userId !== userId && role !== 'ADMIN') {
  throw new ForbiddenException('Acesso negado');
}
await this.prisma.review.delete({ where: { id } });
```

Impacto
Remoção em massa de avaliações (dano à reputação da loja e à prova social) sem rastro de auditoria, contornando a 2FA.

Sugestão de correção
- Em `ReviewsService.remove`, permitir apenas o autor (`review.userId === userId`), sem exceção por papel.
- Criar `DELETE /admin/reviews/:id` com `@AdminOnly()` para moderação (2FA + auditoria automáticas).

Critérios de aceite
- [ ] `DELETE /reviews/:id` com sessão de ADMIN em avaliação de outra pessoa responde 403/404.
- [ ] `DELETE /admin/reviews/:id` sem cookie `naicotech_admin` válido responde 403 `ADMIN_2FA_REQUIRED`.
- [ ] Moderação bem-sucedida cria uma linha em `AdminAuditLog`.
- [ ] O autor continua conseguindo apagar a própria avaliação.
- [ ] Testes e2e cobrindo os três casos acima.
```

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

```
# [Segurança] GET /orders/:id libera pedido de qualquer cliente para ADMIN sem 2FA

**Labels sugeridas:** `security`, `severity:low`

## Descrição
`OrdersService.findOne` confere o dono do pedido, mas abre uma exceção por papel: uma sessão comum de ADMIN lê o pedido de qualquer cliente, sem a 2ª etapa e sem auditoria. O painel não usa essa rota (usa `/admin/orders`), então a exceção não é necessária.

## Por que é explorável
Com só a senha de um admin, sem o código TOTP, é possível ler pedidos alheios. Atenuante: é preciso conhecer o UUID (v4, não enumerável). Ele aparece, por exemplo, nos links `/pedidos/<id>` dos e-mails de envio/entrega.

## Evidência
`backend/src/orders/orders.service.ts:152-158`
```ts
async findOne(id: string, userId: string, role: string) {
  const order = await this.prisma.order.findUnique({ where: { id }, include: orderInclude, omit: orderOmit });
  if (!order || (order.userId !== userId && role !== 'ADMIN')) {
    throw new NotFoundException('Pedido não encontrado');
  }
  return order;
}
```

Impacto
Leitura de itens, valores, frete, código de rastreio e resumo do endereço de clientes sem passar pela 2FA nem deixar registro.

Sugestão de correção
```

```
- Remover `role !== 'ADMIN'` (e o parâmetro `role`) de `findOne`: a rota de cliente atende só o dono.
- Se o painel precisar do detalhe, criar `GET /admin/orders/:id` com `@AdminOnly()`.

## Critérios de aceite
- [ ] `GET /orders/:id` de outro cliente com sessão de ADMIN responde 404.
- [ ] O dono continua lendo o próprio pedido.
- [ ] Teste e2e cobrindo o caso do ADMIN.
```

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

```
# [Segurança] Validação de startup aceita os segredos de teste commitados e chaves de baixa entropia

**Labels sugeridas:** `security`, `severity:low`

## Descrição
`validateEnv()` rejeita só os placeholders do `.env.example`, além de checar tamanho e formato. Os segredos **públicos** de `backend/test/setup-env.ts` passam em todas as checagens, assim como chaves triviais (`000…0`, `111…1`, `222…2`). Em produção também não exige `PROXY_SECRET` nem `SMTP_HOST` (sem eles, a API aceita acesso direto e o e-mail cai em `localhost:1025`).

Agrupa também os segredos de teste fixos (`setup-env.ts`) e a senha do Postgres efêmero do CI, que são inofensivos sozinhos mas alimentam o risco acima.

## Por que é explorável
Condição: alguém copia os valores dos testes para o `.env` de produção (ex.: "para testar rapidinho"). Com o `JWT_SECRET` público, qualquer pessoa forja cookies `naicotech_session` **e** `naicotech_admin` (typ `admin`) para qualquer usuário. Com as chaves AES públicas, um vazamento do banco revela endereços, segredos TOTP e IPs.

Verificado: o `backend/.env` local atual **não** usa esses valores.

## Evidência
`backend/src/config/env.ts:2`
```ts
const PLACEHOLDERS = ['GERE_UM', 'TROQUE_POR', 'seu-access-token', 'chave-secreta', 'cole-o-token'];
```
`backend/src/config/env.ts:20-31`
```ts
const jwtSecret = required('JWT_SECRET');
if (jwtSecret.length < 32) { fail(...) }
...
if (!/^[0-9a-f]{64}$/i.test(totpKey)) { fail(...) }
```
`backend/test/setup-env.ts:8, 12, 15, 31`
```ts
process.env.JWT_SECRET = 'segredo-de-teste-com-mais-de-32-caracteres-0123456789';
process.env.TOTP_ENCRYPTION_KEY = '0123456789abcdef0123456789abcdef0123456789abcdef0123456789abcdef';
process.env.ADDRESS_ENCRYPTION_KEY = 'fedcba9876543210fedcba9876543210fedcba9876543210fedcba9876543210';
process.env.IP_ENCRYPTION_KEY = '00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff';
```
`.github/workflows/ci.yml:21, 32`: `POSTGRES_PASSWORD: senha-so-do-ci`

Impacto
Se a condição ocorrer: tomada de qualquer conta, inclusive do painel admin com 2FA, e quebra da criptografia dos dados em repouso.

Sugestão de correção
- Exportar os valores de teste numa constante compartilhada e recusá-los em `validateEnv()` (fora de `NODE_ENV=test`).
- Checagem mínima de entropia: por exemplo, recusar chaves com menos de 16 bytes distintos.
- Com `NODE_ENV=production`, exigir `PROXY_SECRET` (quando `DEMO_MODE=true` ou atrás do Vercel) e `SMTP_HOST`.

Critérios de aceite
- [ ] A API não sobe com `NODE_ENV=production` usando qualquer segredo de `setup-env.ts`.
- [ ] A API não sobe com chave AES `000…0` / `111…1`.
- [ ] Em produção, ausência de `SMTP_HOST` (e de `PROXY_SECRET` no modo demo) impede a subida, com mensagem clara.
- [ ] Os testes e2e continuam passando.
```

--- FIM ISSUE 3 ---

--- ISSUE 4 ---

```
# [Segurança] docker-compose publica Postgres e Mailpit de dev em todas as interfaces

**Labels sugeridas:** `security`, `severity:low`

## Descrição
O `docker-compose.yml` publica as portas do Postgres (5433), do SMTP do Mailpit (1025) e da **interface web do Mailpit (8025, sem autenticação)** em `0.0.0.0`. O Mailpit guarda os e-mails de verificação e de redefinição de senha, que contêm tokens de uso único.

## Por que é explorável
Condição: a máquina de desenvolvimento está numa rede compartilhada (Wi-Fi público, coworking) e o firewall permite a conexão. Qualquer pessoa na rede abre `http://<ip-da-máquina>:8025`, pede "esqueci minha senha" para uma conta conhecida e lê o link no Mailpit, tomando a conta (inclusive a de admin local). O Postgres também fica acessível, protegido só pela senha do `.env`.

## Evidência
`docker-compose.yml:11-12`
```yaml
ports:
  - "5433:5432"
```
```

`docker-compose.yml:21-23`
```yaml
ports:
  - "1025:1025"
  - "8025:8025"
```

Impacto
Tomada de contas do ambiente de desenvolvimento e acesso ao banco local (que pode conter dados de teste realistas).

Sugestão de correção
```yaml
ports:
  - "127.0.0.1:5433:5432"
...
ports:
  - "127.0.0.1:1025:1025"
  - "127.0.0.1:8025:8025"
```

Critérios de aceite
- [ ] `docker compose ps` mostra as três portas ligadas em `127.0.0.1`.
- [ ] De outra máquina da rede, as portas 5433, 1025 e 8025 não respondem.
- [ ] Backend, testes e Mailpit continuam funcionando localmente.

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Segurança] Avaliações públicas expõem o nome completo dos compradores

Labels sugeridas: `security`, `severity:low`

Descrição
A rota pública `GET /reviews/product/:productId` devolve `user.name` completo de quem avaliou, e a página do produto exibe esse nome. O resto do projeto segue minimização de dados (endereço mascarado, só o primeiro nome no cabeçalho).

Por que é explorável
Qualquer visitante, sem login, associa o nome completo de uma pessoa à compra de um produto específico. Não é furo de autorização, é exposição de dado pessoal além do necessário (LGPD, art. 6º, III).

Evidência
`backend/src/reviews/reviews.service.ts:42-53`
```ts
findByProduct(productId: string) {
  return this.prisma.review.findMany({
    where: { productId },
    select: {
      id: true, rating: true, comment: true, createdAt: true,
      user: { select: { name: true } },
    },
```
`frontend/src/pages/ProductPage.tsx:217`: `<strong>{r.user.name}</strong>`

Impacto
Exposição pública de nome completo + histórico de compra.

Sugestão de correção
Mapear a resposta para um nome de exibição reduzido (ex.: "Ana S."), sem enviar o nome completo ao navegador.

Critérios de aceite
- [ ] A resposta de `GET /reviews/product/:id` não contém o nome completo.
- [ ] A página do produto mostra o nome reduzido.
- [ ] Teste cobrindo o formato do nome devolvido.

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Segurança] Higiene: e-mail em claro no log e URL da etiqueta sem validação de protocolo

Labels sugeridas: `security`, `severity:info`

Descrição
Dois pontos pequenos de defesa em profundidade:

1. `AuthService.deliver` escreve o e-mail completo do cliente no log quando o envio falha, contrariando a política do próprio projeto (o `MailService` mascara o e-mail).
2. A URL de impressão da etiqueta vem da API do Melhor Envio e é aberta pelo painel com `window.open` sem checar se é `https:`.

Por que é explorável
1. Quem tiver acesso aos logs (provedor de hospedagem, ferramenta de logs) vê e-mails de clientes.
2. Só se a API do Melhor Envio for comprometida e devolver `javascript:`/`data:`. Improvável, mas a checagem é barata.

Evidência
`backend/src/auth/auth.service.ts:243-246`
```ts
private deliver(kind: string, email: string, send: () => Promise<void>) {
  send().catch((error) =>
```

````
    this.logger.error(`Falha ao enviar e-mail de ${kind} para ${email}`, error as Error),
  );
}
```
`backend/src/shipping/shipments.service.ts:165`: `return { url: await this.melhorEnvio.printUrl(order.shipmentId) };`
`frontend/src/pages/admin/OrdersTab.tsx:99`: `window.open(url, '_blank', 'noopener,noreferrer')`

## Impacto
Exposição de dado pessoal em logs; risco residual de execução de script no painel admin.

## Sugestão de correção
- Exportar `maskEmail` do `MailService` (ou um util comum) e usá-lo no log do `AuthService`.
- Em `ShipmentsService.printUrl`, validar `new URL(url).protocol === 'https:'` (e, se quiser, o host do Melhor Envio) antes de devolver.

## Critérios de aceite
- [ ] Nenhum log do backend contém um e-mail completo (grep nos testes de falha de envio).
- [ ] `printUrl` com URL não-https responde erro e não repassa a URL.
- [ ] Teste unitário para cada um dos dois casos.
````

--- FIM ISSUE 6 ---
````